

Лабораторная работа №2

Первоначальна настройка git

Авраменко Давид

Содержание

1	Цель работы	5
2	Задание	6
3	Выполнение лабораторной работы	7
3.1	Установка программного обеспечения	7
3.2	Базовая настройка git	8
3.3	Создание ключей	12
3.4	Настройка автоматических подписей коммитов git	13
3.5	Настройка gh	14
3.6	Создание репозитория курса на основе шаблона	14
3.7	Настройка каталога курса	15
4	Контрольные вопросы	17
5	Выводы	22
6	Список литературы	23

Список иллюстраций

3.1	Установка git	Ошибка! Закладка не определена.
3.2	Установка gh	6
3.3	Установка имени и почты	6
3.4	Настройка utf-8	7
3.5	Генерация ключа	8
3.6	Вывод списка ключей	9
3.7	Вывод списка ключей	9
3.8	Ввод ключа	10
3.9	Готовый ключ	10
3.10	Задание названия ветки и параметры autocrlf и safecrlf	Ошибка! Закладка не определена.
3.11	Создание по алгоритму rsa с ключём размером 4096 бит	11
3.12	Создание по алгоритму ed25519	11

3.13 Настройка автоматических подписей коммитов git	12
3.14 Авторизация	12
3.15 Создание рабочего пространства	13
3.16 Создание рабочего пространства	13
3.17 Создание рабочего пространства	14

Список таблиц

1 Цель работы

Изучить идеологию и применение средств контроля версий, а также освоить базовые умения по работе с git.

2 Задание

- Создать базовую конфигурацию для работы с git.
- Создать ключ SSH. • Создать ключ PGP.
- Настроить подписи git.
- Зарегистрироваться на Github.
- Создать локальный каталог для выполнения заданий по предмету.

3 Выполнение лабораторной работы

3.1 Установка программного обеспечения

Установили git (рис. 3.1).

```
avramenkodavid@fedora:~$ sudo dnf install git -y
[sudo] пароль для avramenkodavid:
Обновление и загрузка репозитория:
Репозитории загружены.
Пакет "git-2.53.0-1.fc44.x86_64" уже установлен.

Нечего делать.
```

Рисунок 3.1: Установка git

Установили gh (рис. 3.2).

```
avramenkodavid@fedora:~$ sudo dnf install gh -y
Обновление и загрузка репозитория:
Репозитории загружены.
Пакет
Установка:
gh
Аrch.
x86_64
Версия
0:2.97.0-2.fc44
Репозиторий
updates
39

Сводка транзакции:
Установка: 1 пакета

Общий размер входящих пакетов составляет 12 MiB. Необходимо загрузить 12 MiB.
После этой операции будут использоваться дополнительные 40 MiB (установка 40 MiB, удаление 0 B).
[1/1] gh-0:2.97.0-2.fc44.x86_64 100% | 1.9 MiB/s | 11.8 MiB |
-----
[1/1] Всего 100% | 1.8 MiB/s | 11.8 MiB | 00m06
Выполнение транзакции
[1/3] Проверить файлы пакета 100% | 18.0 B/s | 1.0 B | 00m00s
[2/3] Подготовить транзакцию 100% | 1.0 B/s | 1.0 B | 00m01s
[3/3] Установка gh-0:2.97.0-2.fc44.x86_64 100% | 18.5 MiB/s | 40.0 MiB | 00m02s
Завершено!
```

Рисунок 3.2: Установка gh

3.2 Базовая настройка git

Задали имя и email владельца репозитория (рис. 3.3).

```
avramenkodavid@fedora:~$ git config --global user.name "David Avramenko"
avramenkodavid@fedora:~$ git config --global user.email "avramenkodavid22@gmail.com"
```

Рисунок 3.3: Установка имени и почты

Настроили utf-8 в выводе сообщений git (рис. 3.4).

```
avramenkodavid@fedora:~$ git config --global core.quotePath false
```

Рисунок 3.4: Настройка utf-8

Настроили верификацию и подписание коммитов git. Для этого сгенерировали ключ(рис. 3.5).

```

avramenkodavid@fedora:~$ gpg --full-generate-key
gpg (GnuPG) 2.4.9; Copyright (C) 2025 g10 Code GmbH
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

gpg: создан каталог '/home/avramenkodavid/.gnupg'
Выберите тип ключа:
  (1) RSA and RSA
  (2) DSA and Elgamal
  (3) DSA (sign only)
  (4) RSA (sign only)
  (9) ECC (sign and encrypt) *default*
 (10) ECC (только для подписи)
 (14) Existing key from card
Ваш выбор? 1
длина ключей RSA может быть от 1024 до 4096.
Какой размер ключа Вам необходим? (3072)
Запрошенный размер ключа - 3072 бит
Выберите срок действия ключа.
  0 = не ограничен
  <n> = срок действия ключа - n дней
  <n>w = срок действия ключа - n недель
  <n>m = срок действия ключа - n месяцев
  <n>y = срок действия ключа - n лет
Срок действия ключа? (0) 0
Срок действия ключа не ограничен
Все верно? (y/N) y

Введите идентификатор пользователя для идентификации ключа.

Avramenko David Andreevich
i почты: avramenkodavid22@gmail.com

```

Рисунок 3.5: Генерация ключа

Из предложенных опций выбрали:

- тип RSA and RSA;
- размер 4096;
- срок действия не истекает никогда;
- Имя (Avramenko David Andreevich);
- Адрес электронной почты (avramenkodavid22@gmail.com).

Вывели список ключей и скопировали отпечаток приватного ключа(в моем случае 277D8DFE26A062E1)(рис. 3.6).

```
avramenkodavid@fedora:~$ gpg --list-secret-keys --keyid-format=long
gpg: проверка таблицы доверия
gpg: marginals needed: 3 completes needed: 1 trust model: pgp
gpg: глубина: 0 достоверных: 1 подписанных: 0 доверие: 0-, 0q, 0n, 0m, 0f, 1u
[keyboard]
-----
sec   rsa3072/277D8DFE26A062E1 2026-08-22 [SC]
      4BE486B6D8BAA2A7653DEEDC277D8DFE26A062E1
uid   [абсолютно] Avramenko David Andreevich <avramenkodavid22@gmail.com>
      ECD36A7DEFA501 2026-08-22 [E]
```

Рисунок 3.6: Вывод списка ключей

Вывели ключ в формате ASCII по его отпечатку и скопировали его(рис. 3.7).

```
avramenkodavid@fedora:~$ gpg --armor --export 277D8DFE26A062E1 > ~/pgp-key.txt
avramenkodavid@fedora:~$ cat ~/pgp-key.txt
-----BEGIN PGP PUBLIC KEY BLOCK-----

mQGNBGqJ4T0BDADyd2bFwErTDPUvdBWaNRwVzRrLZYhv75UfdU8n7ugKJ54GLDqg
NhvU9pSpeDGfFxf9Y15Y+WN7M0ucEH7vQzA2PkYsd6pX+/TDNIJ25F9gP4TkX095
ktJG6bXCbZX8Arx0aqsKZaXrXxEz94M4SWzR/GVmeIDu9aBfLGSGR0bMtF952jgd
UVRQ7BwI9GjyDNVIarMoGDA6Vhdk3pmpzHxhAK2HnTvKHS9ADpDNE5rSrhJ/qzyz
2M0wQbFrrzZ4f64oA1Pc4JE3b/3q72/RQ6BIw9D9pRtQg0T3vPiS3Hiv90Yfyxc7
BWZ2SDQnvv1QY51gXkEe8tjrpStAzQ/C/Rizp616qq65nZHB74Ir+Sa5DJWB6TDj
Dv3f23pnT9XrcjAYcPvv2uh0sIn6u4q5n5e0XzBCX4WfP+ggVmgWW461uPLzJ8B+
5m5JymJwPBBjeK7Id7oxTvwjhSL3HAF4Xq2a6vfvD0kWRSx3t8dx8EAMskALP93j
ijfcUdji5Uw6WHMAEQEAAbQ3QXZyYw1lbmtvIERhdmLkIEFuZHJlZXZpY2ggPGF2
cmFtZW5rb2RhdmLkMjJAZ21haWwY29tPokB0QQAQoA0xYhBEvkhrbYUqKnZT3u
3Cd9jf4moGLhBQJqieE9AhsDBQsJCAcCAiICBhUKCQgLAgQWAgMBAh4HAheAAoJ
ECd9jf4moGLhBQkMALtZsRgNBCPreqQ4zzX6MspDSwvM+mwWLAq9U5Vwq7fF5pnC
997aLH1QKe7fbr1b0ZQdqCxaP/YJ5a9hSTq1K8shXYmTpCuFC3nG822DWmi9AbFi
w2HTv0RPIDE1u08VsrjllAspUZfbb+1nf1NgfQBoMmZsFEogXFXHhiVpuDBWwnzz
4sVB/KUBUewyG+FWtJSChHqyCfo/QS6g4FfxV960UVFSPf
qMJEHm/FVZPdN9aciNE+eRQ130AmRSq0x8b3s1KrQYXE140
mG+jb8WRFc9quiDSC2VzM/UpiuHdWMBhdqlrnyiUpMvtbuj
Vfq0YciBiU84VcS0jOLPE+Q8mgCyRbXZMDQKQV/1x0Hmiw
5KahUbzv0ubX30g/D2ukhuQIT897cdMQ5PRQBk1kxv0s34I
bbkBJQRqieE9AQwAwSzavtjl7I21E3xgAanAwp8T+heWtfU7
```

Рисунок 3.7: Вывод списка ключей

Перешли в настройки GitHub (<https://github.com/settings/keys>), нажали на кнопку New GPG key и вставили полученный ключ в поле ввода, после чего получили готовый загруженный ключ(рис. 3.8),(рис. 3.9).

Add new GPG key

Title

Мой gpg ключ для лаб

Key

-----BEGIN PGP PUBLIC KEY BLOCK-----

```
mQGNBGqJ4T0BDADyd2bFwErTDPuVdBWaNRwVzRrLZYhv75UfdU8n7ugKJ54GLDqg
NhvU9pSpeDGfFxq9Y15Y+WN7M0ucEH7vQzA2PkYsd6pX+/TDNIJ25F9gP4TkXO95
ktjGbXCbZX8ArxOaqsbKzaXrXxEz94M4SWzR/GVmeIDu9aBflGSGRObMtF952jgd
UVRQ7BwI9GjyDjIvIarMoGDA6Vhdk3pmpzHxhAK2HnTvKHS9ADpDNE5rSrhj/qzyz
2MOwQbFrrzZ4f64oA1Pc4JE3b/3q72/RQ6BIw9D9pRtQg0T3vPiS3Hiv90Yfyxc7
BWZ2SDQnwwlQY51gXkEe8tjrpStAzQ/C/Rizp61Gqq65nZHB74Ir+Sa5DJWB6TDj
Dv3f23pnT9XrcjAYcPwv2uh0sIn6u4q5n5e0XzBCX4WFp+ggVmgWW461uPLzJ8B+
```

Add GPG key

Рисунок 3.8: Ввод ключа

GPG keys

This is a list of GPG keys associated with your account. Remove any keys that you do not recognize.



GPG

Мой gpg ключ для лаб

Email address: avramenkodavid22@gmail.com

Key ID: 277D8DFE26A062E1

Subkeys: 24ECD36A7DEFA501

Added on Aug 22, 2026

Learn how to [generate a GPG key and add it to your account.](#)

Рисунок 3.9: Готовый ключ

Возвращаемся к базовой настройке git. Задали имя начальной ветки (будем называть её master) и параметры autocrlf и safecrlf.

3.3 Создание ключей

По алгоритму rsa с ключём размером 4096 бит создали ключ SSH (рис. 3.11).

```

avramenkodavid@fedora:~$ ssh-keygen -t rsa -b 4096 -C "avramenkodavid22@gmail.com"
Generating public/private rsa key pair.
Enter file in which to save the key (/home/avramenkodavid/.ssh/id_rsa):
Created directory '/home/avramenkodavid/.ssh'.
Enter passphrase for "/home/avramenkodavid/.ssh/id_rsa" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/avramenkodavid/.ssh/id_rsa
Your public key has been saved in /home/avramenkodavid/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:z+IsVVHs0f09o9V1c30/L0dJIE+0oiu6jqdgBQ0ror4 avramenkodavid22@gmail.com
The key's randomart image is:
+---[RSA 4096]-----+
|.      o.. |
|..      . o.o |
|.o      o...oo|
|. o      .o.+ .o|
|o      S... = o X|
|          o|
|          +|
|          o|
|          |
|          |
+---+
fedora:~$

```

Рисунок 3.11: Создание по алгоритму rsa с ключём размером 4096 бит

По алгоритму ed25519 создали ключ SSH (рис. 3.12).

```

avramenkodavid@fedora:~$ ssh-keygen -t rsa -b 4096 -C "avramenkodavid22@gmail.com"
Generating public/private rsa key pair.
Enter file in which to save the key (/home/avramenkodavid/.ssh/id_rsa):
Created directory '/home/avramenkodavid/.ssh'.
Enter passphrase for "/home/avramenkodavid/.ssh/id_rsa" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/avramenkodavid/.ssh/id_rsa
Your public key has been saved in /home/avramenkodavid/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:z+IsVVHs0f09o9V1c30/L0dJIE+0oiu6jqdgBQ0ror4 avramenkodavid22@gmail.com
The key's randomart image is:
+---[RSA 4096]-----+
|.      o.. |
|..      . o.o |
|.o      o...oo|
|. o      .o.+ .o|
|o      S... = o X|
|          o|
|          +|
|          o|
|          |
|          |
+---+
fedora:~$

```

Рисунок 3.12: Создание по алгоритму ed25519

Ключ PGP уже был создан и добавлен в GitHub во время настройки верификацию и подписание коммитов git.

3.4 Настройка автоматических подписей коммитов git

Используя введённый email, указали Git применять его при подписи коммитов (рис. 3.13).

```
sdora:~$ git config --global user.signinkey 277D8DFE26A062E1
sdora:~$ git config --global commit.gpgsign true
sdora:~$ gpg --armor --export 277D8DFE26A062E1 > ~/pgp-key.txt
sdora:~$
```

Рисунок 3.13: Настройка автоматических подписей коммитов git

3.5 Настройка gh

Для начала авторизовались с помощью команды `gh auth login` (рис. 3.14).

```
avramenkodavid@fedora:~$ gh auth login
? Where do you use GitHub? GitHub.com
What is your preferred protocol for Git operations on this host? SSH
Upload your SSH public key to your GitHub account? /home/avramenkodavid/.ssh/
Title for your SSH key: GitHub CLI
How would you like to authenticate GitHub CLI? Login with a web browser

First copy your one-time code: 9D77-E707
Press Enter to open https://github.com/login/device in your browser...
```

Рисунок 3.14: Авторизация

3.6 Создание репозитория курса на основе шаблона

Создали по шаблону рабочее пространство. На GitHub я уже создала необходимый репозиторий, поэтому необходимо только клонировать его на свою виртуальную машину я (рис. 3.15).

```

avramenkodavid@fedora:~$ git clone git@github.com:
Клонирование в «study_2025-2026_os-intro»...
remote: Enumerating objects: 41, done.
remote: Counting objects: 100% (41/41), done.
remote: Compressing objects: 100% (39/39), done.
remote: Total 41 (delta 1), reused 33 (delta 1), p
Получение объектов: 100% (41/41), 25.55 KiB | 25.5
Определение изменений: 100% (1/1), готово.

```

Рисунок 3.15: Создание рабочего пространства

3.7 Настройка каталога курса

Перешли в каталог курса, удалили лишние файлы и создали необходимые каталоги (рис. 3.16).

```

avramenkodavid@fedora:~$ cd study_2025-2026_os-intro
avramenkodavid@fedora:~/study_2025-2026_os-intro$ echo os-intro > COURSE
avramenkodavid@fedora:~/study_2025-2026_os-intro$ make prepare
Подмодуль «template/presentation» (https://github.com/yamadharma/academic-presentation-markdown-template.git) зарегистрирован п
ти «template/presentation»
Подмодуль «template/report» (https://github.com/yamadharma/academic-laboratory-report-template.git) зарегистрирован по пути «te
te/report»
Клонирование в «/home/avramenkodavid/study_2025-2026_os-intro/template/presentation»...
Клонирование в «/home/avramenkodavid/study_2025-2026_os-intro/template/report»...
le path 'template/presentation': checked out '60a6091ebd9b01a8d98f2c5fae0f98dc63ad031'
le path 'template/report': checked out '708b8c51acd94d4b78e7634198f221e5dec6355'
nizing submodule url for 'template/presentation'
nizing submodule url for 'template/report'
kodavid@fedora:~/study_2025-2026_os-intro$

```

Рисунок 3.16: Создание рабочего пространства

Отправили файлы на сервер (рис. 3.17).

```

avramenkodavid@fedora:~/study_2025-2026_os-intro$ git push
Перечисление объектов: 100%, готово.
Подсчет объектов: 100% (109/109), готово.
При сжатии изменений используется до 4 потоков
Сжатие объектов: 100% (90/90), готово.
Запись объектов: 100% (106/106), 705.46 KiB | 5.30 MiB/s, готово.
Total 106 (delta 41), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (41/41), completed with 1 local object.
To github.com:avramenkodavid22-dotcom/study_2025-2026_os-intro.git
 4a65363..14aa825 master -> master
avramenkodavid@fedora:~/study_2025-2026_os-intro$ echo "Проверка PGP подписи" > test-pgp.txt
avramenkodavid@fedora:~/study_2025-2026_os-intro$ git add test-pgp.txt
avramenkodavid@fedora:~/study_2025-2026_os-intro$ git commit -m "test: проверка PGP подписи"
[master 6a0970f] test: проверка PGP подписи
 1 file changed, 1 insertion(+)
 create mode 100644 test-pgp.txt
avramenkodavid@fedora:~/study_2025-2026_os-intro$ git log --show-signature -1
commit 6a0970fb763eb8819a4e8986471539aa5d1d0cf1 (HEAD -> master)
 gpg: Подпись сделана C6 22 авг 2026 22:21:43 MSK
 gpg:          ключом RSA с идентификатором 4BE486B6D88AA2A7653DEEDC277D8DFE26A062E1
 йствительная подпись пользователя "Avramenko David Andreevich <avramenkodavid22@gmail.com>" [абсолютное]
    Avramenko David Andreevich <avramenkodavid22@gmail.com>
    Sat Aug 22 22:21:43 2026 +0300

t: проверка PGP подписи
kodavid@fedora:~/study_2025-2026_os-intro$

```

Рисунок 3.17: Создание рабочего пространства

4 Контрольные вопросы

1. Что такое системы контроля версий (VCS) и для решения каких задач они предназначены?

Система контроля версий (Version Control System, VCS) представляет собой программное обеспечение, которое позволяет отслеживать изменения в документах, при необходимости производить их откат, определять, кто и когда внес исправления и т.п.

2. Объясните следующие понятия VCS и их отношения: хранилище, commit, история, рабочая копия.

хранилище - это репозиторий, в котором хранятся все файлы и документы, включая историю изменений. commit - отслеживание и сохранение изменений. история - сохраняет в себе изменения проекта на всех этапах. рабочая копия - копия проекта, связанная с репозиторием.

3. Что представляют собой и чем отличаются централизованные и децентрализованные VCS? Приведите примеры VCS каждого вида.

Централизованные VCS: В централизованных VCS весь код и его история хранятся в одном центральном репозитории. Разработчики работают с копией основного репозитория на своих локальных машинах, откуда отправляют изменения в центральное хранилище.

Примеры: - Subversion (SVN): Один из популярных централизованных VCS. Разработчики могут коммитить изменения в центральный репозиторий и обновлять свои локальные копии. - Perforce: Еще один пример системы контроля версий с централизованным подходом, который широко применяется в больших коммерческих проектах.

Децентрализованные VCS: Децентрализованные VCS позволяют каждому участнику проекта иметь полноценную копию всего репозитория. Это означает, что разработчики имеют доступ ко всей истории проекта локально и могут работать независимо от подключения к сети.

Примеры: - Git: Самая популярная децентрализованная система контроля версий. Разработчики могут коммитить, откатывать изменения и создавать ветки без необходимости доступа к центральному серверу. - Mercurial: Еще один пример децентрализованной VCS, обеспечивающий высокую скорость работы и гибкость в управлении кодом.

4. Опишите действия с VCS при единоличной работе с хранилищем.

При единоличной работе с хранилищем (репозиторием) в системе контроля версий (VCS), разработчик ведет работу над кодом самостоятельно без коллективного взаимодействия. В такой ситуации основной упор делается на сохранение версий кода и отслеживание изменений для личного удобства и безопасности. Вот основные действия, которые могут выполняться при единоличной работе с хранилищем:

- Инициализация репозитория
- Клонирование репозитория
- Коммит изменений
- Просмотр истории изменений
- Создание веток
- Обновление репозитория
- Удаление, перемещение файлов
- Откат изменений
- Игнорирование файлов
- Резервное копирование

5. Опишите порядок работы с общим хранилищем VCS.

Работа с общим хранилищем (репозиторием) в системе контроля версий (VCS) включает в себя совместную работу нескольких разработчиков над одним проектом. Вот порядок действий при работе с общим хранилищем VCS:

- Создание или клонирование репозитория
- Получение последних изменений
- Внесение изменений
- Коммит
- Отправка изменений (push)
- Работа с веткам и т.д.

6. Каковы основные задачи, решаемые инструментальным средством git?

Основные задачи, решаемые инструментальным средством Git, включают в себя управление версиями кода, обеспечение совместной работы над проектами, отслеживание изменений, создание и объединение ветвей разработки, а также возможность отката к предыдущим версиям кода. Git также предоставляет возможность создания резервных копий (backup) и управление изменениями в коде, что делает его ключевым инструментом для разработчиков программного обеспечения.

7. Назовите и дайте краткую характеристику командам git.

- git init - создание основного дерева репозитория
- git pull - получение обновлений (изменений) текущего дерева из центрального репозитория
- git push - отправка всех произведённых изменений локального дерева в центральный репозиторий
- git status - просмотр списка изменённых файлов в текущей директории
- git diff - просмотр текущих изменений
- git add . - добавить все изменённые и/или созданные файлы и/или каталоги

- `git add имена_файлов` - добавить конкретные изменённые и/или созданные файлы и/или каталоги
- `git rm имена_файлов` - удалить файл и/или каталог из индекса репозитория (при этом файл и/или каталог остаётся в локальной директории)
- `git commit -am „Описание коммита“` - сохранить все добавленные изменения и все изменённые файлы
- `git commit` - сохранить добавленные изменения с внесением комментария через встроенный редактор
- `git checkout -b имя_ветки` - создание новой ветки, базирующейся на текущей • `git checkout имя_ветки` - переключение на некоторую ветку
- `git push origin имя_ветки` - отправка изменений конкретной ветки в центральный репозиторий
- `git merge --no-ff имя_ветки` - слияние ветки с текущим деревом
- `git branch -d имя_ветки` - удаление локальной уже слитой с основным деревом ветки
- `git branch -D имя_ветки` принудительное удаление локальной ветки
- `git push origin :имя_ветки` удаление ветки с центрального репозитория

8. Приведите примеры использования при работе с локальным и удалённым репозиториями.

Git pull: Команда `git pull` используется для извлечения и загрузки содержимого из удаленного репозитория и немедленного обновления локального репозитория этим содержимым. Слияние удаленных вышестоящих изменений в локальный репозиторий — это обычное дело в процессе совместной работы на основе Git.

9. Что такое и зачем могут быть нужны ветви (branches)?

Ветви (branches) в контексте систем контроля версий, таких как Git, представляют собой параллельные линии разработки, которые позволяют команде разработчиков работать над отдельными фрагментами кода независимо друг от друга. Ветви могут

быть полезны для разработки новых функций, исправления ошибок, экспериментов с кодом и поддержания стабильной основной версии программного обеспечения. Создание и использование ветвей помогает упростить процесс разработки, избегая конфликтов и обеспечивая возможность параллельной работы над различными задачами.

10. Как и зачем можно игнорировать некоторые файлы при commit?

Игнорируемый файл — файл, явным образом помеченный для Git как файл, который необходимо игнорировать.

Игнорируемые файлы — это, как правило, артефакты сборки и файлы, генерируемые машиной из исходных файлов в вашем репозитории, либо файлы, которые по какой-либо иной причине не должны попадать в коммиты.

Вот некоторые распространенные примеры таких файлов (например: `/bin`, `.lock`, `.tmp`, `/packages`)

Игнорируемые файлы отслеживаются в специальном файле `.gitignore`, который регистрируется в корневом каталоге репозитория. В Git нет специальной команды для указания игнорируемых файлов: вместо этого необходимо вручную отредактировать файл `.gitignore`, чтобы указать в нем новые файлы, которые должны быть проигнорированы. Файлы `.gitignore` содержат шаблоны, которые сопоставляются с именами файлов в репозитории для определения необходимости игнорировать эти файлы.

5 Выводы

В ходе выполнения лабораторной работы мы изучили идеологию и применение средств контроля версий, а также освоили умения по работе с git.

6 Список литературы

1. Лабораторная работа №2 [Электронный ресурс] URL: <https://esytem.rudn.ru/mod/page/view.php>
2. GitHub [Электронный ресурс] URL: <https://github.com/>